

A full CRUD API in 15 lines

A serializer, a viewset and a router give you list, create, read, update and delete.

The big idea

An **API** is a set of web addresses that programs use to read and change your data. It sends back JSON, not HTML pages.

For each kind of thing (books, tasks, users) clients want the same jobs: list, add, look at one, change, delete. That's **CRUD**.

DRF bundles them, like `df.describe()` does many standard sums in one call.

What the tests set up

Before each test they log in and create two books. Then they send **real HTTP requests** to your URLs.

```
Book.objects.create(title="Dune", author="Frank Herbert", published_year=1965)
Book.objects.create(title="Emma", author="Jane Austen", published_year=1815)

# The Book model is already written (sandbox/models.py):
#   title, author, published_year (may be empty), plus an automatic id
```

What the tests send and expect

GET /books/	200, both books: Dune first, then Emma (test 1)
GET /books/1/	One book with keys <code>id</code> , <code>title</code> , <code>author</code> , <code>published_year</code> (test 2)
POST /books/	Title "Beloved": 201, saved, reply has its new <code>id</code> (test 3)
POST /books/	Only an author: 400, error names <code>title</code> (test 4)
PATCH /books/1/	<code>{"published_year": 1966}</code> : 200, title still Dune (test 5)
DELETE /books/2/	204, Emma is gone from the database (test 6)
GET /books/999/	404, no such book (test 7)

What the replies look like

Reply bodies for test 1 and test 4. You write none of this logic yourself.

```
# GET /books/  
[  
  {"id": 1, "title": "Dune", "author": "Frank Herbert", "published_year": 1965},  
  {"id": 2, "title": "Emma", "author": "Jane Austen", "published_year": 1815},  
]  
  
# POST /books/ with {"author": "Anonymous"}  
{"title": ["This field is required."]}
```

New words

resource

One kind of thing your API manages, like books.

CRUD

Create, Read, Update, Delete: the basic jobs on data.

collection URL

The address for the whole list: [/books/](#).

detail URL

The address for one item, by id: [/books/1/](#).

status code

[200](#) OK, [201](#) created, [204](#) done, [400](#) bad input, [404](#) not found.

ViewSet

One class that handles all the actions for one resource.

router

Looks at a ViewSet and makes the URLs for it.

Tool: Book.objects.order_by("id")

A **method** on `Book.objects` that gives a sorted **queryset**; a `-` sorts backwards.

```
matilda = Book.objects.create(
    title="Matilda", author="Roald Dahl", published_year=1988
)
Book.objects.create(title="Coraline", author="Neil Gaiman")
print(matilda.id, matilda.title, matilda.published_year)
print(Book.objects.order_by("-id"))
```

OUTPUT

```
1 Matilda 1988
<QuerySet [<Book: Coraline>, <Book: Matilda>]>
```

Tool: ModelSerializer

It reads field types from the model; `class Meta` says which model and fields.

```
class ShortBookSerializer(serializers.ModelSerializer):
    class Meta:
        model = Book
        fields = ["title", "author"]

matilda = Book.objects.create(title="Matilda", author="Roald Dahl")
print(ShortBookSerializer(matilda).data)
```

OUTPUT

```
{'title': 'Matilda', 'author': 'Roald Dahl'}
```

Tool: ModelViewSet

No methods, just two class attributes: which rows, and which serializer.

```
class NewestFirstViewSet(viewsets.ModelViewSet):  
    queryset = Book.objects.order_by("-id")      # which rows  
    serializer_class = ShortBookSerializer      # the class itself, no ()
```

Tool: DefaultRouter

```
demo = DefaultRouter()
demo.register("newest", NewestFirstViewSet, basename="newest")
for url in demo.urls:
    print(url.pattern, "->", url.name)
```

OUTPUT

```
^newest/$ -> newest-list
^newest\.(?P<format>[a-z0-9]+)/?$ -> newest-list
^newest/(?P<pk>[^/\.]+)/$ -> newest-detail
^newest/(?P<pk>[^/\.+)\.(?P<format>[a-z0-9]+)/?$ -> newest-detail
-> api-root
<drf_format_suffix:format> -> api-root
```

Collection URL vs detail URL

/books/ (the whole list)

- GET → **list** all books
- POST → **create** a new book
- No id in the address

/books/{id}/ (one book)

- GET → **retrieve** that book
- PUT → **update**: send every field
- PATCH → **partial update**: only fields sent
- DELETE → **destroy**, reply 204

Your task, step by step

- 1 BookSerializer: a `class Meta` with `model = Book` and four fields → tests 2-4
- 2 BookViewSet: set `queryset` (ordered by `id`) and `serializer_class` → tests 1, 5-7
- 3 `router = DefaultRouter()`, then register at `"books"`, `basename="book"`
- 4 Set `urlpatterns = router.urls`, after the register line → tests 1-7

Common mistakes

- `Content-Type ... text/html` or `200 != 201`: `urlpatterns` is still empty
- Every test gives `404`: the prefix isn't exactly `"books"`
- Prefix `"/books/"`: write `"books"`, the router adds slashes
- `serializer_class = BookSerializer()`: give the class, no brackets
- Books in the wrong order: the queryset needs `order_by("id")`

Where you'll use this

A reading-list phone app needs to show, add, edit and remove books. With these ~15 lines, your server answers all of it at `/books/` and `/books/{id}/`.

- Most DRF APIs are built this way: model, serializer, viewset, router
- Next exercises add rules to this setup: who may do what, then filtering
- `DefaultRouter` also gives you a browsable front page listing your resources

Check yourself

1. Which URL and method does a router send to `partial_update`?
2. Why does the viewset's queryset need `order_by("id")`?
3. What's the difference between `PUT` and `PATCH`?
4. Why does a successful `DELETE` reply `204` with no body?

Keep learning

Quickstart

Django REST framework · tutorial · beginner. Start here: a working API with a serializer, a viewset and a router in a few minutes, the same three pieces as this exercise.

Tutorial 6: ViewSets & Routers

Django REST framework · tutorial · beginner. Explains how a viewset replaces separate list and detail views and how the router writes the URLs for you.

Serializers: ModelSerializer

Django REST framework · docs · intermediate. Reference for ModelSerializer and its Meta fields option, which builds serializer fields from your model.

ViewSets: ModelViewSet

Django REST framework · docs · intermediate. Lists the actions ModelViewSet gives you (list, create, retrieve, update, partial_update, destroy).

Routers: DefaultRouter

Django REST framework · docs · intermediate. Shows the exact URL patterns DefaultRouter generates, useful when a test hits a URL you did not expect.